

SQL Statements Field Guide

A compact reference for CMPIF2110 and the SQL you are most likely to meet next.

Course foundation acknowledged: The Module 3 Part 1 materials introduce USE, SELECT, FROM, WHERE, DISTINCT, LIKE, BETWEEN, ORDER BY, comparison operators, NULL tests, comments, and semicolon terminators. Examples here use MySQL 8.0 when syntax differs across database systems.

SECTION 01

Basic Statements

The everyday verbs for choosing, reading, adding, changing, and removing data.

USE

USE chooses the database that receives later statements in the current session.

UTILITY

```
USE demo;
```

SELECT / FROM

SELECT names the columns to return, while FROM names the table that supplies them.

DQL

```
SELECT contact_name, city
FROM customer;
```

SELECT DISTINCT

DISTINCT removes duplicate combinations from the selected columns without changing stored data.

DQL

```
SELECT DISTINCT city
FROM customer;
```

WHERE

WHERE keeps only rows whose condition evaluates as true.

DQL

```
SELECT * FROM product
WHERE unit_price > 25;
```

ORDER BY

ORDER BY sorts results by one or more columns using ascending or descending order.

DQL

```
SELECT product_name, unit_price FROM product
ORDER BY unit_price DESC;
```

INSERT

INSERT adds one or more new rows to a table.

DML

```
INSERT INTO category (category_name)
VALUES ('Produce');
```

UPDATE

UPDATE changes existing rows selected by its WHERE condition.

DML

```
UPDATE product SET unit_price = 27.50
WHERE product_id = 12;
```

DELETE

DELETE removes rows selected by its WHERE condition while leaving the table itself intact.

DML

```
DELETE FROM customer
WHERE customer_id = 91;
```

SECTION 02

Intermediate Statements

Structure data, connect tables, summarize groups, and package reusable queries.

CREATE TABLE

DDL

CREATE TABLE defines a new table, its columns, data types, keys, and constraints.

```
CREATE TABLE category (  
  category_id INT PRIMARY KEY AUTO_INCREMENT, category_name VARCHAR(50) NOT NULL  
);
```

ALTER TABLE

DDL

ALTER TABLE changes the columns, constraints, indexes, or other definition details of an existing table.

```
ALTER TABLE product  
ADD COLUMN discontinued BOOLEAN DEFAULT FALSE;
```

DROP TABLE

DDL

DROP TABLE permanently removes a table definition and the data stored in it.

```
DROP TABLE old_import;
```

INNER JOIN

DQL

INNER JOIN returns only rows whose join keys match in both tables.

```
SELECT o.order_id, c.contact_name  
FROM orders o INNER JOIN customer c ON c.customer_id = o.customer_id;
```

LEFT JOIN

DQL

LEFT JOIN keeps every row from the left table and supplies NULL when the right table has no match.

```
SELECT c.category_name, p.product_name  
FROM category c LEFT JOIN product p ON p.category_id = c.category_id;
```

GROUP BY

DQL

GROUP BY collapses rows with shared values so aggregate functions can summarize each group.

```
SELECT category_id, COUNT(*) AS product_count  
FROM product GROUP BY category_id;
```

HAVING

DQL

HAVING filters completed groups after aggregation, unlike WHERE which filters rows first.

```
SELECT category_id, AVG(unit_price) AS avg_price  
FROM product GROUP BY category_id HAVING AVG(unit_price) > 25;
```

SUBQUERY

DQL

A subquery nests one query inside another to supply a value, list, or temporary result set.

```
SELECT product_name FROM product  
WHERE unit_price > (SELECT AVG(unit_price) FROM product);
```

CREATE VIEW

DDL

CREATE VIEW saves a named query that can be read like a virtual table.

```
CREATE VIEW active_products AS  
SELECT * FROM product WHERE discontinued = FALSE;
```

SECTION 03

Advanced Statements

Common industry tools for readable analysis, ranked results, safe changes, and access control.

WITH (CTE)

DQL

WITH creates a named temporary result that makes a complex query easier to read and reuse.

```
WITH totals AS (SELECT customer_id, SUM(order_total) total FROM orders GROUP BY customer_id)
SELECT * FROM totals WHERE total > 1000;
```

CASE

DQL

CASE applies conditional logic inside a query and returns the value from the first matching branch.

```
SELECT product_name, CASE WHEN unit_price >= 50 THEN 'Premium' ELSE 'Standard' END AS tier
FROM product;
```

ROW_NUMBER

DQL

ROW_NUMBER assigns a sequence within an ordered result without collapsing its rows.

```
SELECT product_name, unit_price,
ROW_NUMBER() OVER (ORDER BY unit_price DESC) AS price_rank FROM product;
```

UNION / UNION ALL

DQL

UNION combines compatible query results and removes duplicates, while UNION ALL keeps them.

```
SELECT city FROM customer_usa UNION SELECT city FROM customer_canada;
SELECT city FROM customer_usa UNION ALL SELECT city FROM customer_canada;
```

EXISTS

DQL

EXISTS tests whether a related subquery returns at least one row.

```
SELECT c.contact_name FROM customer c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

TRANSACTIONS

TCL

START TRANSACTION groups changes on transactional tables so COMMIT saves them and ROLLBACK cancels them unless a statement causes an implicit commit.

```
START TRANSACTION; UPDATE product SET unit_price = 30 WHERE product_id = 12; COMMIT;
START TRANSACTION; DELETE FROM orders WHERE order_id = 99; ROLLBACK;
```

CREATE INDEX

DDL

CREATE INDEX builds a lookup structure that can speed common searches at the cost of storage and write work.

```
CREATE INDEX idx_product_category
ON product (category_id);
```

GRANT / REVOKE

DCL

GRANT gives privileges to an account and REVOKE removes privileges already assigned.

```
GRANT SELECT ON demo.* TO 'analyst'@'localhost';
REVOKE SELECT ON demo.* FROM 'analyst'@'localhost';
```

SECTION 04

Operations

The operators and functions that turn a statement into a useful question or calculation.

COMPARISON

Compares values for equality, inequality, or relative order for use in a condition.

```
SELECT * FROM product
WHERE unit_price >= 25 AND unit_price <> 50;
```

LOGICAL

Combines or reverses conditions with AND, OR, and NOT.

```
SELECT * FROM customer
WHERE country = 'USA' AND NOT city = 'Boston';
```

LIKE

Matches text patterns where percent means any sequence and underscore means one character.

```
SELECT DISTINCT city FROM customer
WHERE city LIKE 'B%';
```

BETWEEN

Checks whether a value falls inside an inclusive lower and upper boundary.

```
SELECT * FROM product
WHERE unit_price BETWEEN 25 AND 50;
```

IN

Checks whether a value matches any member of a supplied list or subquery.

```
SELECT * FROM customer
WHERE country IN ('USA', 'Canada', 'Mexico');
```

NULL TESTS

Uses IS NULL or IS NOT NULL because NULL represents an unknown or missing value rather than an ordinary value.

```
SELECT * FROM employee
WHERE supervisor_id IS NULL;
```

ARITHMETIC

Calculates numeric results with addition, subtraction, multiplication, division, or remainder operators.

```
SELECT quantity, price,
quantity * price AS extended_price FROM order_detail;
```

AGGREGATES

Summarizes rows with COUNT, SUM, AVG, MIN, and MAX.

```
SELECT COUNT(*) AS products,
AVG(unit_price) AS avg_price,
MIN(unit_price) AS low, MAX(unit_price) AS high
FROM product;
```

STRING FUNCTIONS

Transforms text with common functions such as CONCAT, UPPER, LOWER, and TRIM.

```
SELECT CONCAT(UPPER(TRIM(last_name)),
', ', first_name) AS display_name FROM employee;
```

DATE FUNCTIONS

Returns and calculates date values with functions such as CURRENT_DATE, DATE_ADD, and DATEDIFF.

```
SELECT order_id, DATEDIFF(CURRENT_DATE, order_date)
AS days_old FROM orders;
```

COALESCE

Returns the first non-NULL value in its argument list and supplies a readable fallback when needed.

```
SELECT employee_name,
COALESCE(supervisor_name, 'None') AS supervisor
FROM employee;
```

LIMIT / OFFSET

Returns a requested slice of an ordered result for previews or pagination in MySQL.

```
SELECT product_name, unit_price FROM product
ORDER BY unit_price DESC LIMIT 10 OFFSET 20;
```